DevOps engineers ki troubleshooting ante enti, andhulo logic + problem-solving ela use avtayi ane examples cheptha – real-world style lo.

🔧 1. CI/CD Pipeline Suddenly Fails — Deep Real-Time Troubleshooting

Situation
Morning varaku Jenkins pipeline successful ga run ayyindi.
Ippudu developer code push chesadu.
Pipeline:
Git
↓
Jenkins
↓
Maven Build
↓
Unit Tests
↓
SonarQube
↓
Docker Build
↓
ECR/Nexus
↓
Deploy
But pipeline suddenly FAILED.
Developer:
"Bro, pipeline fail ayyindi. Fix cheyyi."
❌ Beginner approach
"Jenkins problem emo."
Jenkins restart chestadu.
Again run chestadu.
Fail.
Malli restart.
This is troubleshooting kaadu. This is guessing.

🧠 Correct DevOps approach
Step 1 — First identify EXACTLY where it failed
Jenkins console output open cheyyi.
Suppose:
Stage: Checkout SUCCESS
Stage: Build SUCCESS
Stage: Test FAILED
Stage: SonarQube NOT EXECUTED
Stage: Docker NOT EXECUTED
Ippudu manaki first important information dorikindi:
Jenkins itself completely down kaadu.
Because checkout and build already succeeded.
So Jenkins ni suspect list nunchi almost remove cheyyachu.
Logic:
Checkout SUCCESS
Build SUCCESS
Test FAILED

Therefore:
Jenkins agent is probably working
Git access is working
Build tool is working
Failure is likely around TEST stage

Step 2 — Exact error choodu
Test stage lo:
Tests run: 25
Failures: 0

Errors: 1

Connection refused: localhost:5432
Ippudu "test failed" ane broad information kanna,
"localhost:5432 connection refused"
ane information important.

Step 3 — Error ni understand cheyyi
Application test database ki connect avvadaniki try chesthundi.
Jenkins Test
↓
localhost:5432
↓
Database
Connection refused ante usually:
Aa address/port daggara service listening lo ledu.
Immediately:
"Database password wrong"
ani assume cheyyakudadhu.
Why?
Wrong password ayithe usually authentication error vastundi.
Connection refused ante network/service/listener level issue suspect cheyyali.

Step 4 — Possible causes list prepare cheyyi
Now hypothesis list:
Possibility 1
Database process running lo ledu.
Possibility 2
Wrong hostname/port configured.
Possibility 3
CI environment lo database available ledu.
Possibility 4
Test expects local DB but Jenkins agent lo DB ledu.
Possibility 5
Database container start avvaledu.
Possibility 6
Recent configuration change jarigindi.
Ippudu random ga fix cheyyakunda one by one eliminate cheyyali.

Step 5 — Jenkins environment understand cheyyi
Developer local machine lo:
Application → localhost:5432 → PostgreSQL
Jenkins agent lo:
Application → localhost:5432 → ???
Important question:
Jenkins lo localhost ante developer laptop kaadu. Jenkins agent machine/container.
This is a very common real-world troubleshooting concept.
Local lo DB running undachu.
Jenkins lo DB running undakapovachu.
So:
Works locally
≠
Works in CI

Step 6 — Recent change check cheyyi
Suppose developer yesterday application-test.properties change chesadu.
Before:
db.host=test-db
db.port=5432
Now:
db.host=localhost
db.port=5432
Aha!

Jenkins environment lo localhost DB ledu.
Root cause:
Configuration changed from test-db to localhost.
Not Jenkins.
Not Maven.
Not SonarQube.
Not Docker.
Test configuration problem.

Step 7 — Fix
Correct test configuration restore cheyyali.
Or better architecture:
Jenkins
↓
Test Container / Test DB
↓
Application Tests
CI pipeline isolated test database use cheyyachu.

Step 8 — Verify
Fix chesaka pipeline rerun.
Expected:
Checkout SUCCESS
Build SUCCESS
Test SUCCESS
SonarQube SUCCESS
Docker SUCCESS
Push SUCCESS
Deploy SUCCESS
But troubleshooting ikkade finish kaadu.

Step 9 — Prevention
Question:
"Future lo same problem malli rakunda em cheyyali?"
Possible improvements:

- CI-specific configuration maintain cheyyali.
- Test DB automatically create cheyyali.
- Environment variables use cheyyali.
- Integration tests ki isolated database use cheyyali.
- Pipeline lo clear error messages maintain cheyyali.
- Configuration changes review cheyyali.

This is root-cause thinking.

🐬 2. Docker Container Repeatedly Crashing — Deep Approach
Situation:
docker ps
Container kanipinchadu.
So:
docker ps -a
Output:
myapp Exited (1)
First thought:
Container crash ayyindi.
But why?
Exited (1) root cause kaadu.

Step 1 — Logs
docker logs myapp
Suppose:
Error: DATABASE_URL environment variable is missing
Now suspect:
Configuration.

Step 2 — Container configuration inspect
docker inspect myapp
Check:

- Environment variables
- Mounts
- Network
- Ports
- Image
- Command

Suppose:
DATABASE_URL = not present
Now root cause almost clear.

Step 3 — Why is variable missing?
Check:
docker-compose.yml
.env
Jenkins credentials
Kubernetes Secret
Suppose compose file:
environment:
 DB_HOST: postgres
 DB_PORT: 5432
But application expects:
DATABASE_URL
Root cause:
Application expects one variable.
Container provides another.
So:
Application expectation
 ↓
DATABASE_URL

Container configuration
 ↓
DB_HOST + DB_PORT
Mismatch.

Step 4 — Fix
Either application configuration change cheyyali or correct environment variable provide cheyyali.
Then:
docker compose up
Again test.

☁️ 3. Works Locally, Fails in Production — Deep Approach
This is one of the most important DevOps troubleshooting scenarios.
Suppose developer says:
"Na laptop lo perfectly working bro."
Production lo:
HTTP 500
Don't respond:
"Code problem."
First compare environments.

Layer 1 — Version
Check:
Application version
Java version
Node version
Python version
Docker image version

Dependency versions
Example:
Local:
Java 17
Production:
Java 11
Possible compatibility issue.

Layer 2 — Configuration
Local:
DB_HOST=localhost
Production:
DB_HOST=prod-db.internal
Expected behavior different.

Layer 3 — Secrets
Local developer machine may have:
AWS credentials
DB password
API keys
Production container may not.
Then application fails.

Layer 4 — Network
Local:
Laptop → DB
Production:
EC2 → Security Group → DB
Security Group block chesthe application DB connect avvadu.

Layer 5 — Filesystem
Local Linux/Mac/Windows differences kuda matter avvachu.
Example:
config.json
vs
Config.json
Linux case-sensitive.

Layer 6 — Resource limits
Production container:
Memory limit = 512 MB
Application needs:
1 GB
Then OOMKill avvachu.

🐬 4. Container Running But Application Not Accessible
This is another important distinction.
Suppose:
docker ps
shows:
myapp Up 10 minutes
Developer:
"Container running undi. But browser lo app open avvatledu."
Don't conclude Docker is fine.
Container running ≠ application reachable.

Step 1 — Check application inside container
docker exec -it myapp sh
Then:
curl localhost:8080
Case A:
Response vastundi.
Then application itself is probably working.

Now check:
Docker port mapping
Firewall
Security Group
Load Balancer
Nginx
DNS
Case B:
curl localhost:8080 fails.
Then problem is inside container/application.
This is powerful troubleshooting logic:
Outside fails
↓
Test from inside

Inside works?
↓ YES → Network/Port/Proxy
↓ NO → Application/Container

🌐 5. Website Down — Layer-by-Layer
User says:
"Website open avvatledu."
Don't immediately SSH into EC2.
Follow request path:
Browser
↓
DNS
↓
Internet
↓
Load Balancer
↓
Security Group
↓
Nginx
↓
Application
↓
Database
Check each layer.
DNS
Does domain resolve?
Load Balancer
Are targets healthy?
Security Group
Is required traffic allowed?
Nginx
Is service running?
Application
Is process listening?
Database
Is application dependency healthy?
This is called layered troubleshooting.

📈 6. CPU 95% — Don't Just Restart
Suppose alert:
CPU = 95%
Bad troubleshooting:
reboot
Why?
Because reboot may hide the evidence.

Step 1

top
Find top CPU process.
Suppose:
java → 92%
Now problem narrowed to Java process.

Step 2
Ask:
Why is Java using 92%?
Possibilities:

- Traffic increased
- Infinite loop
- Garbage collection
- Bad deployment
- Thread issue
- Query problem
- Background job

Step 3
Check application metrics/logs.
Suppose at exactly 2 AM:
CPU ↑
and:
Backup job started
Then maybe scheduled job caused load.

💾 7. Disk 100% — Deep Logic
Alert:
/dev/xvda1 100%
First:
df -h
Now know filesystem.
Next:
du -sh /*
Suppose:
/var = 80 GB
Then:
du -sh /var/*
Suppose:
/var/log = 75 GB
Then:
Which log?
Maybe:
application.log = 70 GB
Now ask:
Why did log grow to 70 GB?
Maybe:
Error happening repeatedly
So don't just delete the log.
Because actual root cause may be:
Application error
↓
Repeated error
↓
Huge logs
↓
Disk full
↓
Application failure
If you only delete the log:
Disk free
but:

Application error still exists
After few hours disk fills again.
That's the difference between temporary fix and root-cause fix.

☸️ 8. Kubernetes CrashLoopBackOff — Real Thinking
Pod:
myapp-xxx CrashLoopBackOff
Don't Google only the word "CrashLoopBackOff".
It means roughly:
Container starts, exits/crashes, Kubernetes retries, and backs off between restarts.
Now ask:
Why did container exit?
kubectl logs <pod>
Suppose:
Connection refused to database
Then check DB.
But if logs show:
Missing environment variable
check Secret/ConfigMap.
If logs show nothing:
kubectl describe pod <pod>
Look at Events.
Maybe:
OOMKilled
Then it's memory-related.

🚦 9. Kubernetes Pod Pending
Different from CrashLoopBackOff.
Pending
means pod may not even have been successfully scheduled.
Check:
kubectl describe pod <pod>
Suppose:
0/3 nodes are available:
Insufficient cpu
Now:
Application issue? ❌
Container image issue? ❌
Scheduling/resource issue? ✅
This is why status alone isn't enough.

🔐 10. Database Connection Failure — Deep Layering
Application:
Could not connect to database
Possible causes are many.
Don't change password immediately.
Follow:
1. DNS
Can application resolve DB hostname?
2. Network
Can server reach DB network?
3. Port
Is DB port reachable?
4. Database service
Is DB running?
5. Authentication
Username/password correct?
6. Authorization
Does user have required permissions?
7. Application configuration
Correct DB URL?
So:

DNS
↓
Network
↓
Port
↓
Service
↓
Authentication
↓
Authorization
↓
Application
This layered model is very important for real troubleshooting.

🔥 11. Deployment Caused Production Issue
Before deployment:
Error rate = 0.2%
After deployment:
Error rate = 8%
This is a strong correlation.
Now:
New deployment
↓
Metrics changed
↓
Logs changed
↓
Compare previous version
If confirmed:
Rollback first if business impact is high.
Then investigate.
Important DevOps thinking:
Production lo users suffer avutunte:
First restore service. Then investigate deeply.
Don't spend 2 hours proving root cause while production is down.

🧠 12. Intermittent Problem — Hardest Type
Suppose:
"API sometimes works, sometimes gives 500."
This is harder than a permanent failure.
You need to compare:
Successful Request
vs
Failed Request
Check:

- Timestamp
- Request ID
- Server/pod
- Database
- Response time
- CPU/memory
- Logs
- External API

Suppose 3 Kubernetes pods:
Pod A → works
Pod B → works
Pod C → fails
Aha!
Now problem is probably pod-specific.
Check Pod C configuration/image/environment.
This is why correlation is powerful.

📊 13. Monitoring Alert — How to Think
Prometheus says:
CPU > 90%
Don't immediately treat it as outage.
Ask:
CPU high?
↓
Application latency high?
↓
Error rate high?
↓
User impact?
If:
CPU = 95%
Latency = normal
Errors = normal
Maybe it's a legitimate workload spike.
If:
CPU = 95%
Latency = 10 sec
Errors = 20%
Now it's a real incident.
So monitoring should use multiple signals, not one metric alone.

🧩 14. The Most Important Skill: Compare
Real-world troubleshooting lo comparison is extremely powerful.
Compare:
Working vs failing
Working server
vs
Failing server
Old vs new
Old deployment
vs
New deployment
Local vs production
Local
vs
Production
Healthy pod vs unhealthy pod
Pod A
vs
Pod B
Successful request vs failed request
Request #101 → SUCCESS
Request #102 → FAIL
Then ask:
What is different?
That difference is often your clue.

🧠 15. Don't Jump Directly to Root Cause
Suppose:
Website down
Possible causes:
DNS
Load Balancer
Network
Nginx
Application
Database
Don't say:
"Database problem."

Instead:
Hypothesis → Test → Result
Example:
Hypothesis: DB is down.
Test: Can application connect to DB?
Result: Yes.
Therefore:
DB hypothesis eliminated.
Next:
Hypothesis: Nginx issue.
Test: Curl backend directly.
Result: Backend works.
Therefore:
Nginx/proxy path becomes more suspicious.
This is deductive troubleshooting.

🚨 16. Production Incident — Exact Sequence
Suppose 2 AM:
PagerDuty/Alert
↓
Website 500 errors
Phase 1 — Detect
Confirm:
Is it really happening?
Phase 2 — Scope
Ask:
All users?
One region?
One API?
One service?
Phase 3 — Impact
How many users?
How severe?
Phase 4 — Recent changes
Deployment?
Configuration?
Infrastructure?
Database migration?
Phase 5 — Evidence
Collect:
Logs
Metrics
Traces
Events
Deployment history
Phase 6 — Mitigation
If latest deployment caused issue:
Rollback
Phase 7 — Root Cause
After service restored:
Why did it happen?
Phase 8 — Prevention
Monitoring
Testing
Automation
Documentation
Alert improvement

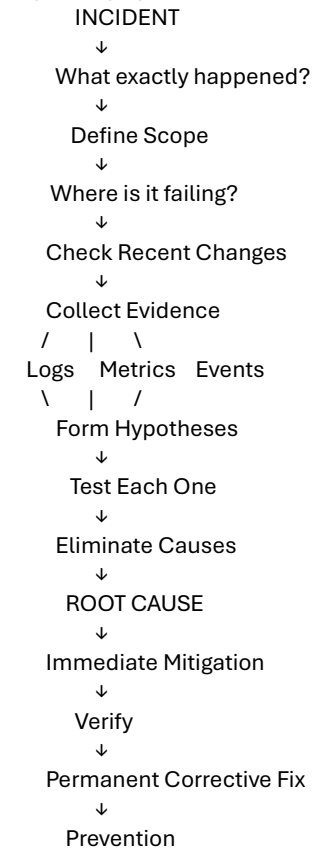
🔥 Real DevOps Troubleshooting Mindset

Nuvvu commands memorize cheyyadam kanna ee questions memorize chesko:

Question 1:
What exactly is failing?
Question 2:
Where is it failing?
Question 3:
When did it start?
Question 4:
What changed before it started?
Question 5:
Can I reproduce it?
Question 6:
What evidence do I have?
Question 7:
What are the possible causes?
Question 8:
How can I eliminate each cause?
Question 9:
What is the safest immediate fix?
Question 10:
How do I prevent it from happening again?

🏆 One Complete Mental Model

Any DevOps problem ni ee flow lo choodu:



//

Part 2 continue

17. Docker Container Starts and Immediately Exits

Situation
docker ps
lo container kanipinchadu.
But:
docker ps -a
lo:
myapp Exited (1)
kanipistundi.
First thought
Container start avvatledu ani anukokudadhu.
Actually container start ayyindi, but main process exit ayyindi.
Docker container ki important concept:
Container lives as long as its main process is running.
Example:
Container starts
  ↓
Application process starts
  ↓
Application crashes
  ↓
Main process exits
  ↓
Container stops
Step 1 — Logs
docker logs myapp
Suppose:
Port 8080 already in use
Now possible causes:

- Another process already using port
- Wrong port configuration
- Multiple containers trying same host port

Step 2 — Check port
docker ps -a
Check port mapping.
Host:
8080
Container:
8080
Suppose another application already occupies host 8080.
Fix
Change mapping:
8081:8080
Meaning:
Host 8081 → Container 8080
Lesson
Exited (1) is only a symptom.
Actual root cause comes from:
Status → Logs → Configuration → Environment → Dependency

🌐 18. DNS Is Not Resolving
Situation
Developer says:
"Application running undi, but domain open avvatledu."
First don't check application.
Check DNS.
Example:
nslookup example.com
or:
dig example.com

Possible results
Case 1 — Correct IP
DNS is probably okay.
Move to:
Load Balancer
Case 2 — Wrong IP
Maybe DNS record points to old server.
Case 3 — No response
Possible:

- DNS record missing
- Wrong hosted zone
- DNS propagation/configuration issue
- Resolver issue

Thinking:
Domain
↓
DNS
↓
IP
↓
Load Balancer
↓
Application
Don't jump directly to EC2.

🔐 19. SSH to EC2 Is Not Working
Situation
ssh ec2-user@server
fails.
Possible error:
Connection timed out
Don't immediately think:
EC2 is down.
Investigation
First ask:
Can I reach the server network-wise?
Then check:

- Correct public/private IP?
- Instance running?
- Security Group allows port 22?
- Network ACL?
- Route table?
- Internet Gateway?
- Corporate VPN/proxy?
- Correct subnet?
- Correct username?
- Correct SSH key?

Important distinction
If:
Connection timed out
usually think network/connectivity path.
If:
Permission denied (publickey)
network may already be working.
Now focus on:

- Username
- SSH key
- Authorized keys
- SSH configuration

Lesson

Error message itself narrows your search space.

🔥 20. Application Gives HTTP 500
Situation
Browser:
500 Internal Server Error
Don't assume
Server is broken.
HTTP 500 generally means server-side application processing failed.
Step 1
Check application logs.
Example:
NullPointerException
Then likely application code/runtime issue.
Another example:
Database connection timeout
Then application may be healthy, but dependency failed.
Another:
Permission denied
Then filesystem/configuration issue.
Thinking:
500
↓
Application logs
↓
Actual exception
↓
Which dependency?
↓
Root cause
HTTP status is not root cause.

🚦 21. HTTP 502 vs 503 vs 504
Very useful in production.
502 — Bad Gateway
Usually proxy/load balancer got an invalid response from upstream.
Think:
Nginx/LB
↓
Backend
Backend may be:

- Down
- Returning invalid response
- Connection failed

---

503 — Service Unavailable
Service currently unavailable.
Think:

- No healthy backend
- Application unavailable
- Kubernetes service has no ready endpoints
- Maintenance/capacity issue

---

504 — Gateway Timeout
Proxy/LB waited too long for backend.
Think:
Client
↓
Load Balancer
↓
Backend
⏳

⏳
⏳
Timeout
Possible:

- Slow DB
- Slow API
- Application overloaded
- Network latency
- Long-running request

Shortcut mental model
502 → Bad response/connectivity to upstream
503 → No healthy service
504 → Upstream took too long

🧠 22. Memory Leak / OOMKilled
Situation
Kubernetes:
OOMKilled
or Linux:
Out of memory
First question
Who consumed memory?
Not:
"Restart pod."
Check:

- Container memory usage
- Application heap
- Number of processes
- Traffic
- Cache
- Memory limits

Example
Normal:
Memory → 400 MB
After 1 hour:
600 MB
After 2 hours:
800 MB
After 3 hours:
950 MB
Then:
1 GB limit → OOMKilled
This pattern suggests a possible memory leak or uncontrolled memory growth.
But don't conclude immediately.
Could also be:

- Traffic increase
- Larger requests
- Cache growth
- JVM configuration
- Insufficient memory limit

Need evidence.

🗄️ 23. Database Is Extremely Slow
Situation
Application response time:
200 ms → 8 seconds
Developer says:
"App code changed ledu."
Good clue.
Now investigate dependency.

Check application metrics
Maybe:
DB response time ↑
Then investigate DB.
Possible:

- Slow query
- Missing index
- Lock contention
- High connections
- CPU high
- Disk I/O high
- Storage issue
- Large table scan

Logic
Application slow
↓
Is DB slow?
/ \
YES NO
↓ ↓
DB Other
investigate dependency
This is dependency isolation.

🔗 24. External API Is Failing
Situation
Your application is healthy, but payment/email/third-party API calls fail.
Logs:
Connection timeout
Check:

- Is external API reachable?
- DNS working?
- Internet/NAT working?
- Firewall?
- Proxy?
- API endpoint correct?
- API credentials valid?
- Rate limit exceeded?
- Third-party outage?

Important
Don't immediately modify your application.
First establish:
Is the failure inside our system or outside our system?
That's a very important production troubleshooting question.

💳 25. API Rate Limit Exceeded
Situation
External service returns:
429 Too Many Requests
This is different from:
500
Your application may be perfectly healthy.
The external service is telling you:
You're sending too many requests.
Investigate:

- Request volume increased?
- Retry loop?
- Multiple application replicas?
- Missing caching?
- Batch job started?

- Rate limit changed?

Dangerous pattern
API fails
↓
Application retries
↓
More requests
↓
Rate limit increases
↓
More retries
↓
System gets worse
This is called a retry storm pattern.

🔄 26. Deployment Is Hanging
Situation
Jenkins:
Deploying...
Deploying...
Deploying...
20 minutes.
Don't cancel immediately.
First identify where it's waiting.
Maybe:
Jenkins
↓
kubectl apply
↓
Kubernetes
↓
Waiting for rollout
Check Kubernetes:
kubectl rollout status deployment/myapp
Maybe:
Waiting for deployment to finish...
Then:
kubectl get pods
Suppose new pods:
0/2 Ready
Now deployment isn't the real problem.
Pods aren't becoming Ready.
Then:
kubectl describe pod
kubectl logs
Follow the chain.

🩺 27. Readiness Probe Failing
Situation
Pod:
Running
but:
READY 0/1
This confuses beginners.
Running ≠ Ready.
Container process can be alive, but Kubernetes may not consider it ready to receive traffic.
Example
Application listens on:
8080
Probe configured:
/health
port 8081
Probe fails.

Result:
Pod Running
Pod Not Ready
Service doesn't send traffic
Root cause
Probe configuration mismatch.

📦 28. ImagePullBackOff
Situation
Kubernetes pod:
ImagePullBackOff
This means Kubernetes is having trouble pulling the container image.
Check:
kubectl describe pod <pod>
Possible causes:

- Image name wrong
- Image tag doesn't exist
- Registry unavailable
- Private registry authentication missing
- ImagePullSecret incorrect
- Network issue

Example
Jenkins pushed:
myapp:v12
Deployment requests:
myapp:v13
v13 doesn't exist.
So:
CI/CD → Registry → Kubernetes
The failure is at image retrieval, not application startup.

🔑 29. Secret Expired
Situation
Everything worked yesterday.
Today:
401 Unauthorized
Possible API credential/token expired.
Don't change application code.
Check:

- Token expiration
- Secret value
- IAM permissions
- Secret rotation
- Application loaded latest secret?
- Secret version?

Important real-world clue
Sudden failure without deployment often points toward:

- Expired credentials
- Certificates
- DNS
- External dependency
- Resource exhaustion
- Scheduled jobs

Not always, but these become useful hypotheses.

🔒 30. SSL/TLS Certificate Expired
Situation
Users see:
NET::ERR_CERT_DATE_INVALID
or API calls fail TLS validation.

Check:

- Certificate expiration
- Correct certificate attached?
- Load balancer certificate?
- Nginx certificate?
- Certificate chain?
- Renewal automation?
- DNS points to correct endpoint?

Important:
Certificate can be valid on one endpoint and expired on another.
So identify:
User
↓
DNS
↓
Endpoint
↓
Certificate actually presented

🕐 31. “It Works Sometimes” Due to Multiple Servers
Situation
Users report:
"Refresh chesthe sometimes works, sometimes fails."
Very interesting.
Suppose load balancer has:
Server A → Healthy
Server B → Healthy
Server C → Broken
Traffic is distributed.
So:
Request 1 → A → SUCCESS
Request 2 → C → FAIL
Request 3 → B → SUCCESS
Request 4 → C → FAIL
User experiences:
"Sometimes works."
Investigation
Compare:
Request ID
Server/Pod
Timestamp
Response
If all failures come from Server C:
Root cause becomes server-specific.
This is one of the most valuable troubleshooting patterns.

🍼 32. “No Changes Happened” — Still Something Changed
Developer:
"Bro, deployment emi cheyyaledu. Suddenly fail ayyindi."
Don't assume nothing changed.
Possible changes outside application deployment:

- Certificate expired
- Secret rotated
- Dependency released new version
- External API changed
- DNS changed
- Cloud resource changed
- Disk filled
- Scheduled job ran
- Traffic increased

- Auto-scaling changed
- Security rule changed

Mental model:
No code change
≠
No change anywhere
Production is a dynamic system.

🧩 33. The 5 Most Important Troubleshooting Dimensions
Any incident lo first these 5 questions ask cheyyi:
1. WHAT?
Exactly what is failing?
Build?
Container?
Network?
Application?
Database?
2. WHERE?
Which component/layer?
Client?
DNS?
LB?
Server?
Container?
DB?
External API?
3. WHEN?
When did it start?
After deployment?
After config change?
At 2 AM?
After traffic spike?
4. WHAT CHANGED?
Code?
Image?
Config?
Secret?
Infrastructure?
Traffic?
External dependency?
5. WHAT IS DIFFERENT?
Working vs failing
Old vs new
Local vs production
Pod A vs Pod B
These five questions alone will massively improve your troubleshooting.

🏆 Real-Time Example — Full Incident
Let's say 2 AM alert:
Production API is returning 500 errors.
Don't panic.
Step 1 — Confirm
Check Grafana:
Error rate = 35%
Definitely real.
Step 2 — Scope
Only:
/payment API
is failing.
Other APIs okay.
Step 3 — Recent changes
Check deployment:
Payment service deployed 10 minutes ago.

Strong clue.
Step 4 — Logs
Database column "payment_status" not found
Interesting.
Step 5 — Database
Check schema.
Production DB doesn't have the new column.
Root cause
Application version expected new DB schema, but DB migration wasn't applied.
New Code
↓
Expects New Column
↓
Production DB
↓
Old Schema
↓
500 Error
Immediate mitigation
If business impact is high:
Rollback application to previous known-good version.
Then permanent fix
Apply migration safely.
Then deploy application again.
Prevention
Pipeline should have:
Build
↓
Test
↓
DB Migration Validation
↓
Deploy
↓
Smoke Test
This is what real troubleshooting + problem-solving looks like.

🧠 Final Rule — Idi mind lo strong ga petko
When something breaks, don't ask:
“Which command should I run?”
First ask:
“What information will reduce my uncertainty?”
Example:
CPU high.
Instead of randomly running 20 commands:
“Which process is consuming CPU?”
So run:
top
Container crashed.
“Why did the process exit?”
Run:
docker logs
Kubernetes pod pending.
“Why didn't scheduler place it?”
Run:
kubectl describe pod
Website down.
“Which layer is failing?”
Start:
DNS → LB → Network → Server → Application → DB
That shift—from “commands memorization” to “question → evidence → hypothesis → test”—is the real DevOps troubleshooting skill.